

ATLS TALKHAND: A Webcam-Based Recognizer for LSM Fingerspelling

Cristobal Longares
Ingeniería en Sistemas Computacionales, ITZ

Abstract

This report documents ATLS TALKHAND, a student project built to recognize fingerspelled vowels from Mexican Sign Language (Lengua de Señas Mexicana, LSM) using a webcam. The recognition side is a Python script that reads hand landmarks with MediaPipe and classifies handshapes through joint-angle thresholds computed with the law of cosines, not a trained classifier. A separate .NET 7 WinForms application hosts the lesson screens the user interacts with, and the two communicate through a plain JSON file written to disk. I go over how the geometry actually works, how the two halves of the application talk to each other, what is and is not covered by the current implementation (five vowels, no full alphabet), and a few things that did not go well, including a bug in an abandoned motion-based detector and some project management mistakes around what actually got committed to version control.

1 Problem Statement

Most introductory material for learning LSM fingerspelling is static: a poster, a PDF, a video. None of it tells you whether your hand is actually forming the letter correctly, you are left comparing your hand to a picture and guessing. The goal of this project was to build something that gives that feedback live, using a normal laptop webcam and no specialized hardware.

I scoped the recognition problem down to vowels first (A, E, I, O, U) because their handshapes are distinct enough from each other, in terms of which fingers are extended versus curled, to be told apart using simple geometric thresholds on joint angles. That let me avoid the much larger effort of collecting a labeled dataset and training a classifier, which would have been necessary to cover a full alphabet where several letters share similar handshapes and are told apart by orientation or motion instead.

2 Methodology and Implementation

2.1 Hand Landmark Detection

The pipeline uses MediaPipe's `Hands` solution to get 21 3D landmarks per detected hand from each webcam frame. I did not train or fine-tune this model, it is used as-is; the actual engineering work in this project sits on top of its output. The main loop configures it as:

```
1 with mp_hands.Hands(  
2     static_image_mode=False,  
3     max_num_hands=2,  
4     min_detection_confidence=0.75) as hands:
```

2.2 From Landmarks to Joint Angles

The function `obtenerAngulos` in `normalizacionCords.py` converts landmark coordinates into joint flexion angles. For each of six tracked joints, pinky, ring finger, middle finger, index finger, and two on the thumb (outer and inner), it takes three landmarks (tip, PIP, and MCP for the four main fingers; TIP, IP/MCP, and wrist for the thumb pair), scales their normalized MediaPipe coordinates by the frame's width and height, and computes the three side lengths of the triangle those three points form using `np.linalg.norm`. The joint angle then follows from the law of cosines:

$$\cos(\theta) = \frac{l_1^2 + l_3^2 - l_2^2}{2 l_1 l_3}$$

where l_1, l_2, l_3 are the pairwise distances between the three landmarks for that joint. The code guards against the two ways this can misbehave: a zero-length side (which would divide by zero) and a ratio that falls outside $[-1, 1]$ (which would make `acos` raise a domain error), defaulting the angle to 0 in either case rather than crashing.

```
1 if l1 and l3 != 0 and -1 < num_den1 < 1:
2     angle1 = round(degrees(abs(acos(num_den1))))
3 else:
4     angle1 = 0
```

One detail worth flagging here: the `return` statement in `obtenerAngulos` sits inside the `for hand_landmarks in results.multi_hand_landmarks` loop. That means the function always returns after processing the first hand MediaPipe finds, even though the calling code configures `max_num_hands=2`. In practice this was never a problem, since fingerspelling only needs one hand, but it does mean the two-hand configuration in `app.py` was never doing anything.

2.3 From Angles to Letters

The main loop (`app.py`) turns the six angles into a binary vector, one entry per finger, marking it "up" if the angle clears a threshold: 125° for the thumb's outer joint, 150° for the thumb's inner joint, and 90° for the remaining four fingers. These thresholds were tuned by testing in front of my own webcam, there is no labeled dataset behind them and no accuracy measurement, which is the main limitation of the whole approach and is discussed further in Section 4.

`condicionales.py` then matches this six-value vector against hardcoded patterns for each vowel. For example:

```
1 def obtenerA(dedos, frame):
2     font = cv2.FONT_HERSHEY_SIMPLEX
3     if dedos == [1, 1, 0, 0, 0, 0]:
4         cv2.rectangle(frame, (0, 0), (100, 100), (255, 255, 255), -1)
5         cv2.putText(frame, 'A', (20, 80), font, 3, (0, 0, 0), 2, cv2.
6             LINE_AA)
7         data = {"valor_booleanoA": True}
8         with open("info.json", "w") as f:
9             json.dump(data, f)
```

Each of the five `obtener*` functions (`obtenerA`, `obtenerE`, `obtenerI`, `obtenerO`, `obtenerU`) follows this same shape: check the finger vector against the pattern for that letter, draw a small overlay box and label on the frame if it matches, and write a boolean flag to `info.json` either way. There is also a combined `condicionalesLetras` function that runs the same five checks in sequence and ends with a comment, `# abecedario`, that I left myself as a reminder that this was meant to eventually cover the full alphabet. It did not get there in this version of the project.

2.4 An Abandoned Attempt at "J"

Unlike the vowels, "J" in LSM fingerspelling involves motion, not just a static handshake. I tried to detect it in `app.py` by tracking the vertical pixel position of the pinky tip across frames and firing a match if it moved more than 30 pixels between two consecutive frames:

```
1 pinky = obtenerAngulos(results, width, height)[1]
2 pinkY = pinky[1] + pinky[0]
3 resta = pinkY - lectura_actual
4 lectura_actual = pinkY
5 if dedos == [0, 0, 1, 0, 0, 0]:
6     if abs(resta) > 30:
7         cv2.rectangle(frame, (0, 0), (100, 100), (0, 0, ), -1)
8         cv2.putText(frame, 'J', (20, 80), font, 3, (0, 0, 0), 2, cv2.
            LINE_AA)
```

This has a bug: the color argument passed to `cv2.rectangle` is `(0, 0, )`, a two-value tuple, where OpenCV expects a three-value BGR color. If this branch had actually triggered during a run, it would have raised an error. This is very likely why the "J" logic never made it out of `app.py` and into `condicionales.py` alongside the vowels that did ship.

2.5 Getting the Result Out of Python

Since the recognition logic runs in a Python process and the lesson UI is a separate .NET application, the two need a way to exchange state. The mechanism here is about as simple as it gets: every time a recognition function runs, it overwrites `ATLS/info.json` with a small JSON object, for example `{"valor_booleanoA": true}` or the placeholder state `{"letra": "x"}` that ships with the repo. There is no locking and no versioning on that file, whichever process reads or writes it last wins. It works for a single-user desktop demo, but it is not something I would reuse for anything with real concurrency.

2.6 Packaging

The Python side is frozen into a standalone executable with `cx_Freeze`, bundling a Python 3.8 interpreter together with OpenCV and MediaPipe's compiled extensions, so it can run on a machine that does not have Python installed. The .NET side is a WinForms application targeting .NET 7 (namespace `ATLS_TALKHAND`, five forms), built separately and shipped as compiled binaries. It embeds IronPython and IronPython.StdLib, and uses `Newtonsoft.Json` to parse `info.json`, so it can react to whatever the Python process last wrote there.

3 Results / How It Behaves

In its current state, the system reliably distinguishes the five vowel handshapes (A, E, I, O, U) from each other in front of a webcam under reasonable lighting, since each corresponds to a distinct combination of extended and curled fingers, and the angle thresholds were tuned against that. It does not recognize the rest of the LSM fingerspelling alphabet, and the one attempt at a motion-based letter ("J") is broken and unused. There is no quantitative accuracy evaluation in this project, no held-out test set, no confusion matrix, this was validated informally by using it myself, which is a limitation I want to be upfront about rather than imply otherwise.

4 Conclusions and Lessons Learned

Framing hand recognition as a geometry problem, angles between three landmarks compared against fixed thresholds, was enough to get five distinct handshapes working without needing to collect and label any training data. That was the right tradeoff for a first version, but it does not scale to the full alphabet, where multiple letters need finer distinctions than "is this finger up or down," and a few need motion. A proper next step would be logging a set of labeled frames per letter (a few hundred each, across different hands, distances and lighting) and fitting a small classifier on the same MediaPipe landmark features, instead of hand-tuning more degree cutoffs.

A few things about how the project was managed also did not go well and are worth naming directly. The repository carries close to 130 MB of committed binaries, including two nearly identical `cx_Freeze` builds with a full bundled Python runtime and OpenCV/MediaPipe extensions, which should have been a build artifact or a release download, not something tracked in git. The .NET/WinForms half of the project never had its source code (the `.cs` and `.resx` files behind its five forms) committed at all, only the compiled `.dll/.exe/.pdb` made it into version control, which means this repository cannot actually rebuild that half of the application from source. And `app.py`, the script that actually wires `normalizacionCords` and `condicionales` together into a webcam loop, was deleted from the tree in the same commit that added the frozen build, so what remains under `Funciones/` is real working code, but no longer a complete, runnable program by itself. All three are straightforward things to fix in a next iteration: a `.gitignore` for build output, committing the WinForms source, and keeping the Python entry point in the tree even after freezing it for distribution.